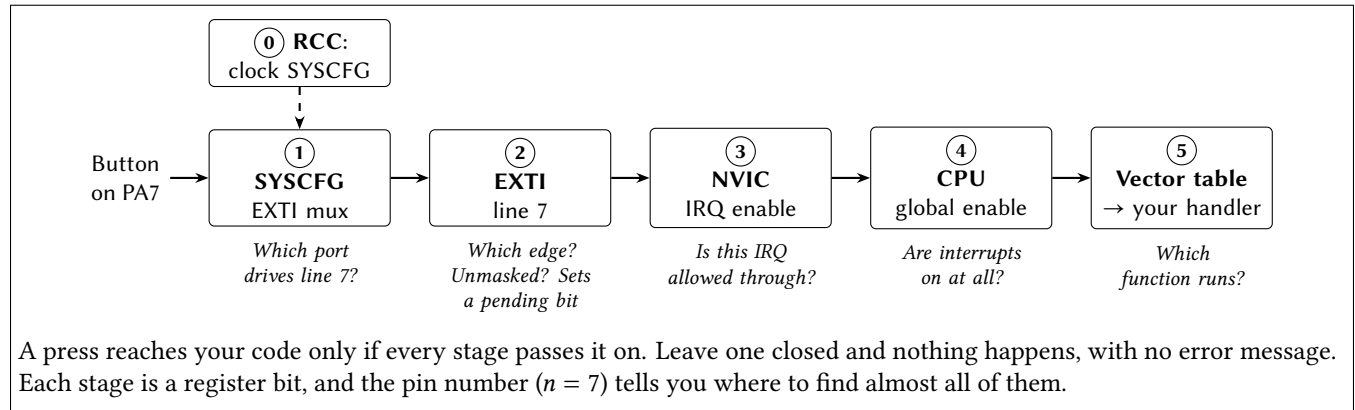


# Interrupt Configuration Activity

Names:

**Goal.** Make a press of the button on PA7 interrupt the CPU, and toggle the LED (PB3) from the interrupt handler instead of polling for the press. Then add the board's second button, on PA4, on your own.

**Starter code:** [github.com/HMC-E155/tutorial-interrupts](https://github.com/HMC-E155/tutorial-interrupts), main branch. **Reference:** RM0394 Ch. 6 (RCC), 9 (SYSCFG), 12 (NVIC and the vector table), 13 (EXTI). PM0214 §4.3 (NVIC registers).



## Part 0: Polling first

1. Flash `button_polling.c` and tap the button quickly several times. Why are some taps missed?

## Part 1: Open each stage for PA7 ( $n = 7$ )

2. Find the interrupt that serves EXTI line 7 in the vector table (RM0394 Ch. 12). What is its name, and what is its position  $p$ ?

For each row of the table, go to the chapter listed, find the register, then use  $n$  (or  $p$  for the NVIC) to pick the bit.

|   | What                       | Look in     | Register | Bit(s) | Value |
|---|----------------------------|-------------|----------|--------|-------|
| ① | Clock SYSCFG               | RM Ch. 6    | _____    | _____  | 1     |
| ① | Connect port A to line 7   | RM Ch. 9    | _____    | _____  | _____ |
| ② | Unmask line 7              | RM Ch. 13   | _____    | _____  | 1     |
|   | No rising-edge trigger     | RM Ch. 13   | _____    | _____  | 0     |
|   | Falling-edge trigger       | RM Ch. 13   | _____    | _____  | 1     |
| ③ | Enable IRQ $p$ in the NVIC | PM0214 §4.3 | _____    | _____  | 1     |
| ④ | Allow interrupts at all    | given       | PRIMASK  | 0      | 0     |

3. Look back at the table. For any pin  $n$ , how do you find the EXTICR register and the bits of its field? For any IRQ position  $p$ , how do you find the ISER register and bit?

4. In the device header (`stm321432xx.h`), register arrays start at 0. How do you write `SYSCFG_EXTICR2` and `NVIC_ISER0` in C?

5. Stage 4 isn't a memory-mapped register. What instruction does `__enable_irq()` run, and what does it change?

Now fill in the TODOs in `main()` of `button_interrupt.c`.

## Part 2: The handler (stage 5)

6. What must the interrupt handler be named? Where does that name come from?

7. Lines 5–9 all share this handler. How does it check that line 7 (the button) caused the interrupt, and how does it clear the request?

**Predict, then try one.** What happens if you...

| Mistake                        | Prediction  |
|--------------------------------|-------------|
| Misspell the handler name      | <div></div> |
| Never clear the pending bit    | <div></div> |
| Skip stage 0 (no SYSCFG clock) | <div></div> |

### Part 3: A second button on PA4 (no hints)

Now do the whole process yourself for the board's other button, on PA4. Make it turn the LED *off*, so you can tell which handler ran. Set PA4 up as an input with a pull-up, as the starter code does for PA7.

8. What are the name and position *p* of EXTI line 4's interrupt?

|   | What                            | Register | Bit(s) | Value |
|---|---------------------------------|----------|--------|-------|
| ① | Connect port A to line 4        | _____    | _____  | _____ |
| ② | Unmask line 4                   | _____    | _____  | _____ |
|   | No rising-edge trigger          | _____    | _____  | _____ |
|   | Falling-edge trigger            | _____    | _____  | _____ |
| ③ | Enable IRQ <i>p</i> in the NVIC | _____    | _____  | 1     |

9. Which stages need nothing new for the second button? Why?

10. Can PA4's code go in EXTI9\_5\_IRQHandler next to PA7's? Write PA4's handler.

11. Could a button on PB7 also interrupt, alongside the one on PA7? Why or why not?